Sebenta 06

Gestão de Rede Centralizada e SDN

Infraestruturas de Computação na Nuvem | Aula 06

Objeto e utilização deste documento

O presente documento acompanha a apresentação da Aula 06. Cada página reproduz um slide e desenvolve, em texto corrido, a matéria nele condensada. A aula é expositiva e retoma dois temas anteriores: os namespaces de rede da Aula 04, que isolam a pilha de rede de cada container, e o cluster Kubernetes da Aula 05, cuja rede de Pods foi usada sem ter sido explicada.

A leitura sequencial é a mais proveitosa. A aula começa pelas limitações da gestão de rede tradicional, apresenta a separação entre plano de controlo e plano de dados e a arquitetura SDN, e detalha o OpenFlow, as tabelas de fluxos e os modelos reativo e proativo. Passa depois pelos controllers e pelo Open vSwitch, pelas redes overlay, pela virtualização de funções de rede e pela microssegmentação, e termina com a concretização destes conceitos no Proxmox e no Kubernetes.

Gestão de Rede Centralizada e SDN

Infraestruturas de Computação na Nuvem, Aula 06

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

A Aula 05 mostrou que o Kubernetes decide onde correm os Pods, mas deixou em aberto uma pergunta: por que caminho chega um pacote de um Pod num nó a um Pod noutro nó? A resposta está na rede, e a rede de um datacenter moderno já não se gere equipamento a equipamento. Gere-se como software.

Esta aula apresenta o modelo que o torna possível. A ideia central é separar quem decide para onde vai o tráfego de quem o encaminha, e entregar a decisão a um elemento centralizado e programável. Daí decorrem o protocolo OpenFlow, os switches virtuais, as redes overlay e as políticas de segurança por carga de trabalho.

Agenda (1/2)

- Limitações da gestão de rede tradicional em datacenters.
- Separação entre plano de controlo e plano de dados.
- Arquitetura de Software-Defined Networking (SDN).
- OpenFlow e tabelas de fluxos.
- Controllers SDN e o modelo reativo vs proativo.

A primeira metade da agenda trata da motivação e do modelo. Parte das limitações da gestão de rede tradicional, explica a distinção entre plano de controlo e plano de dados e descreve a arquitetura da SDN, com os seus três níveis e as duas interfaces que os ligam.

Segue-se o OpenFlow, o protocolo através do qual o controller instala regras nos switches, e a forma como essas regras se organizam em tabelas de fluxos. A metade fecha com a distinção entre os modelos reativo e proativo e com os controllers disponíveis.

Agenda (2/2)

- Open vSwitch e redes de containers/Kubernetes.
- Redes overlay: VLAN, VXLAN e outros encapsulamentos.
- Network Functions Virtualization (NFV).
- Microssegmentação e políticas de rede.
- SDN no Proxmox e no Kubernetes.
- Vantagens e desafios da centralização.

A segunda metade da agenda desce ao concreto. O Open vSwitch é o switch virtual programável que materializa o modelo em hipervisores e em hosts de containers. As redes overlay, em particular a VXLAN, resolvem o problema de criar muitas redes lógicas isoladas sobre a mesma infraestrutura física.

A virtualização de funções de rede e a microssegmentação mostram o que a programabilidade permite fazer com o tráfego: colocar funções em software e aplicar políticas de segurança a cada carga de trabalho. A aula termina com o Proxmox e o Kubernetes, onde estes conceitos já estão presentes, e com as vantagens e os riscos da centralização.



O percurso organiza-se em seis blocos. O primeiro justifica a centralização a partir dos limites do modelo tradicional. O segundo apresenta a separação dos planos e a arquitetura SDN. O terceiro detalha o OpenFlow e as tabelas de fluxos, e o quarto os controllers e o Open vSwitch. O quinto trata das redes overlay, e o sexto reúne a virtualização de funções, as políticas de rede e as plataformas concretas.

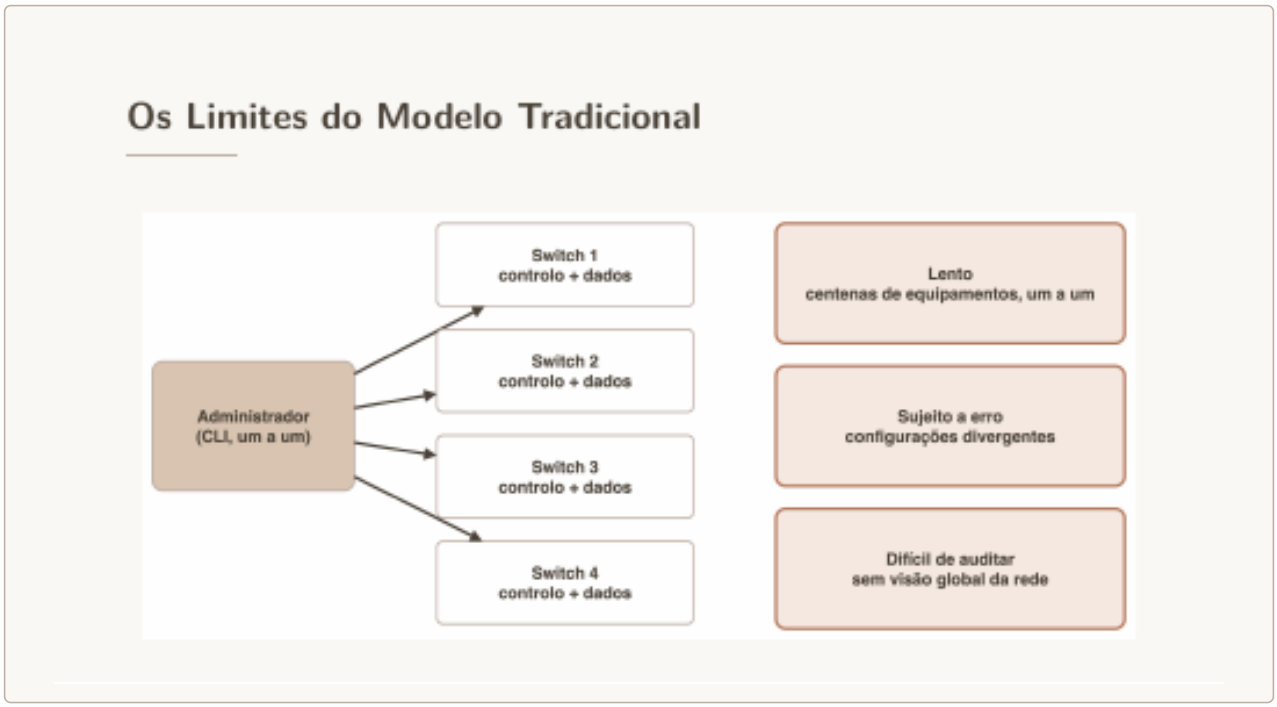
As cores distinguem os blocos de base, a bege, os centrais, a verde, e os que completam o quadro, a branco. Os dois blocos verdes são o núcleo da aula: sem eles, os restantes perdem o contexto.

Gestão de Rede Tradicional

- Cada equipamento de rede (switch, router) tem o seu próprio plano de controlo, configurado individualmente.
- Configuração distribuída, dispositivo a dispositivo, tipicamente via CLI proprietária.
- Em datacenters com centenas ou milhares de switches, este modelo não escala: lento, sujeito a erro humano e difícil de auditar.
- Motivação central da SDN: mover a inteligência de decisão para fora dos equipamentos individuais.

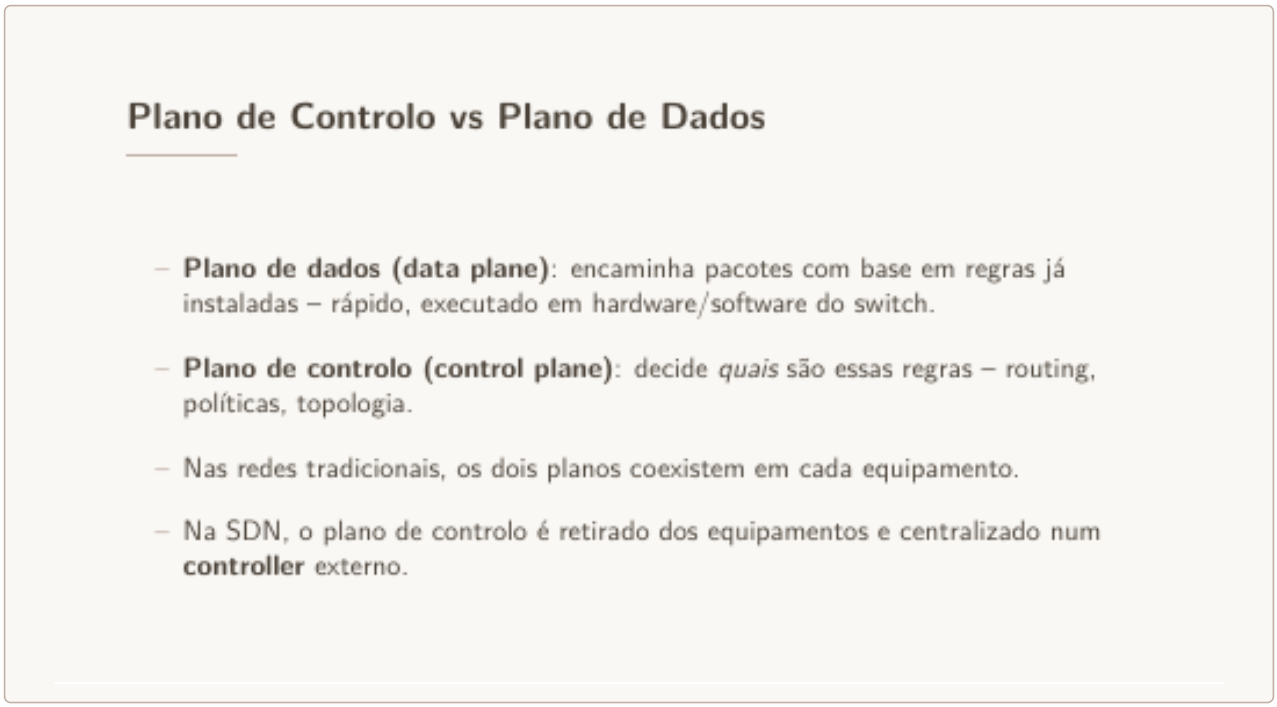
Numa rede tradicional, cada switch e cada router é um equipamento autónomo. Contém o hardware que encaminha pacotes e contém também a inteligência que decide como encaminhá-los: os protocolos de routing, a aprendizagem de endereços, as listas de controlo de acesso. A configuração faz-se equipamento a equipamento, normalmente através de uma interface de linha de comandos própria de cada fabricante.

Esta arquitetura funcionou durante décadas porque as redes eram relativamente estáticas e pequenas. Num datacenter com centenas ou milhares de switches, onde as cargas de trabalho surgem e desaparecem em segundos, deixa de ser adequada. A motivação da SDN é retirar a inteligência de decisão dos equipamentos individuais e geri-la num único ponto.



A figura mostra o administrador a configurar quatro switches, um de cada vez. Cada switch contém o plano de controlo e o plano de dados, e por isso cada um tem de ser configurado e mantido de forma independente.

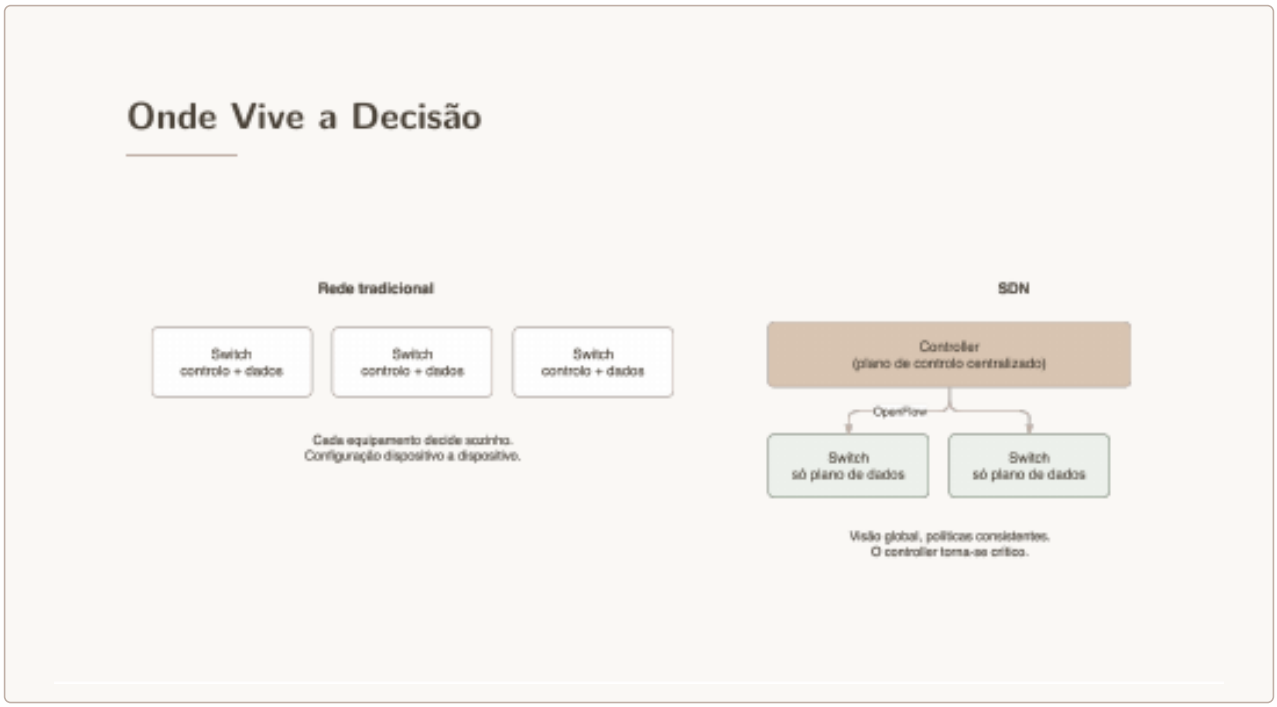
Os três problemas à direita decorrem desta organização. Configurar centenas de equipamentos um a um é lento. Cada intervenção manual é uma oportunidade de erro, e configurações que deviam ser idênticas acabam por divergir. E, sem uma visão global, verificar se a rede cumpre a política pretendida exige percorrer os equipamentos um a um, o que torna a auditoria difícil.



O plano de dados, ou *data plane*, é a parte que encaminha pacotes. Dado um pacote que chega a uma porta, consulta as regras que tem instaladas e decide, em microssegundos, por que porta o deve enviar. Tem de ser rápido, e por isso executa-se em hardware especializado ou em código otimizado do núcleo.

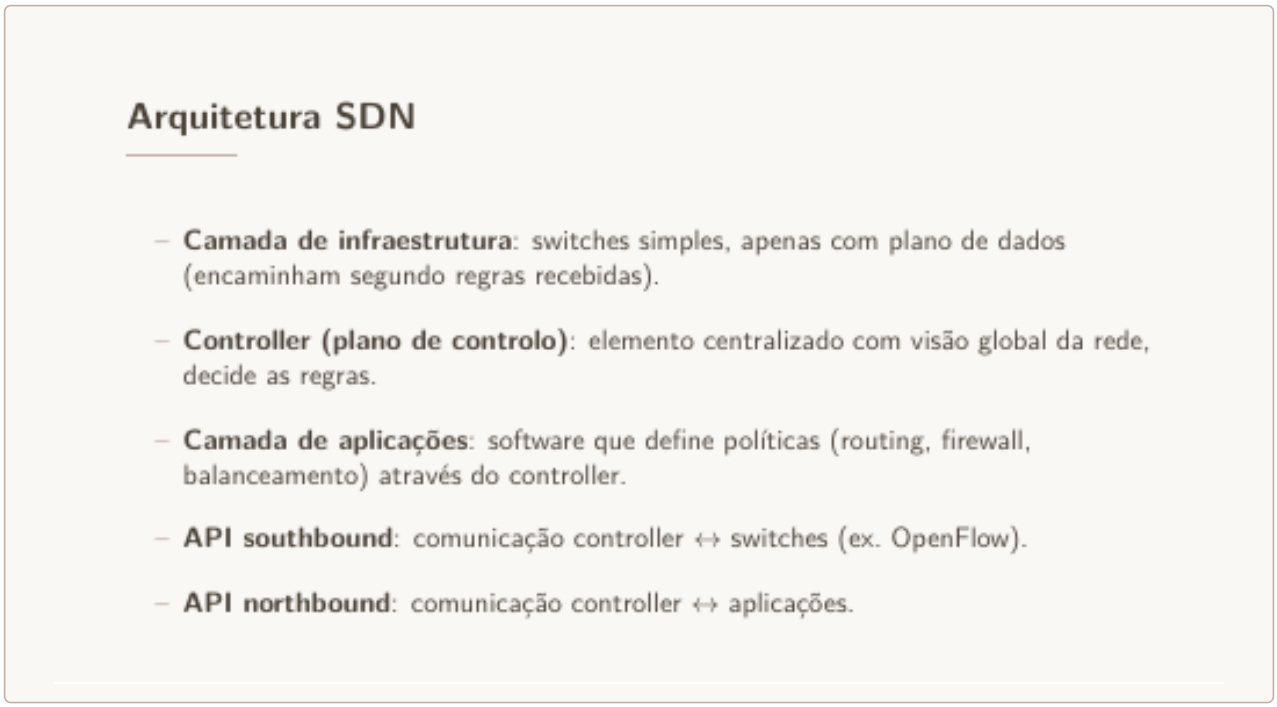
O plano de controlo, ou *control plane*, decide quais são essas regras. Constrói a visão da topologia, calcula os caminhos e define as políticas. Pode ser mais lento, porque atua sobre o conjunto da rede e não sobre cada pacote.

Nas redes tradicionais os dois planos coexistem no mesmo equipamento. Na SDN, o plano de controlo é retirado dos switches e centralizado num *controller*. Os switches ficam reduzidos ao que fazem melhor: aplicar regras a pacotes.



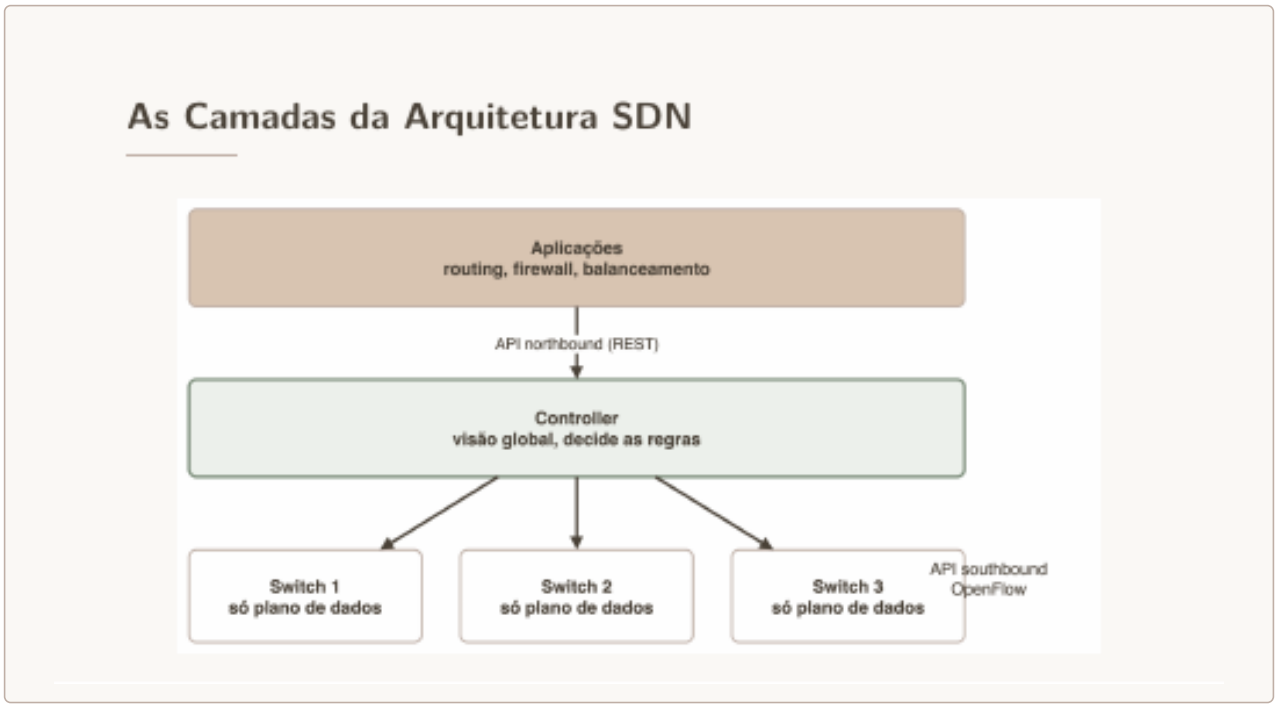
O diagrama coloca lado a lado os dois modelos. À esquerda, a decisão está distribuída: cada equipamento tem a sua inteligência e constrói a sua própria visão da rede através de protocolos de troca de informação. À direita, a decisão está concentrada no controller, que tem uma visão global e se limita a instruir os switches.

A consequência prática é a forma de mudar o comportamento da rede. No modelo tradicional, alterar uma política significa reconfigurar vários equipamentos. No modelo SDN, altera-se a política no controller e este atualiza as regras onde for necessário. A rede passa a ser programável, tal como um servidor.



A arquitetura SDN organiza-se em três níveis. A camada de infraestrutura é formada pelos switches, físicos ou virtuais, que apenas têm plano de dados. O controller, no nível intermédio, é o plano de controlo: conhece a rede inteira e decide as regras. A camada de aplicações contém o software que exprime políticas, como o routing, a firewall ou o balanceamento de carga, e que as entrega ao controller.

Duas interfaces ligam os níveis. A interface *southbound* liga o controller aos switches, e o OpenFlow é o exemplo mais conhecido. A interface *northbound* liga as aplicações ao controller e, em geral, é uma API REST. Os nomes decorrem da forma habitual de desenhar o diagrama, com as aplicações em cima e a infraestrutura em baixo.



O diagrama reproduz as três camadas. A seta superior, a API *northbound*, é o canal pelo qual uma aplicação pede ao controller um comportamento, por exemplo que determinado tráfego seja bloqueado. As setas inferiores, a interface *southbound*, são o canal pelo qual o controller instala as regras correspondentes nos switches.

Convém reparar que a aplicação não fala com os switches. Desconhece o fabricante, o modelo e o protocolo de cada um. É o controller que traduz a intenção em regras concretas, e por isso a mesma aplicação funciona sobre equipamento diferente.

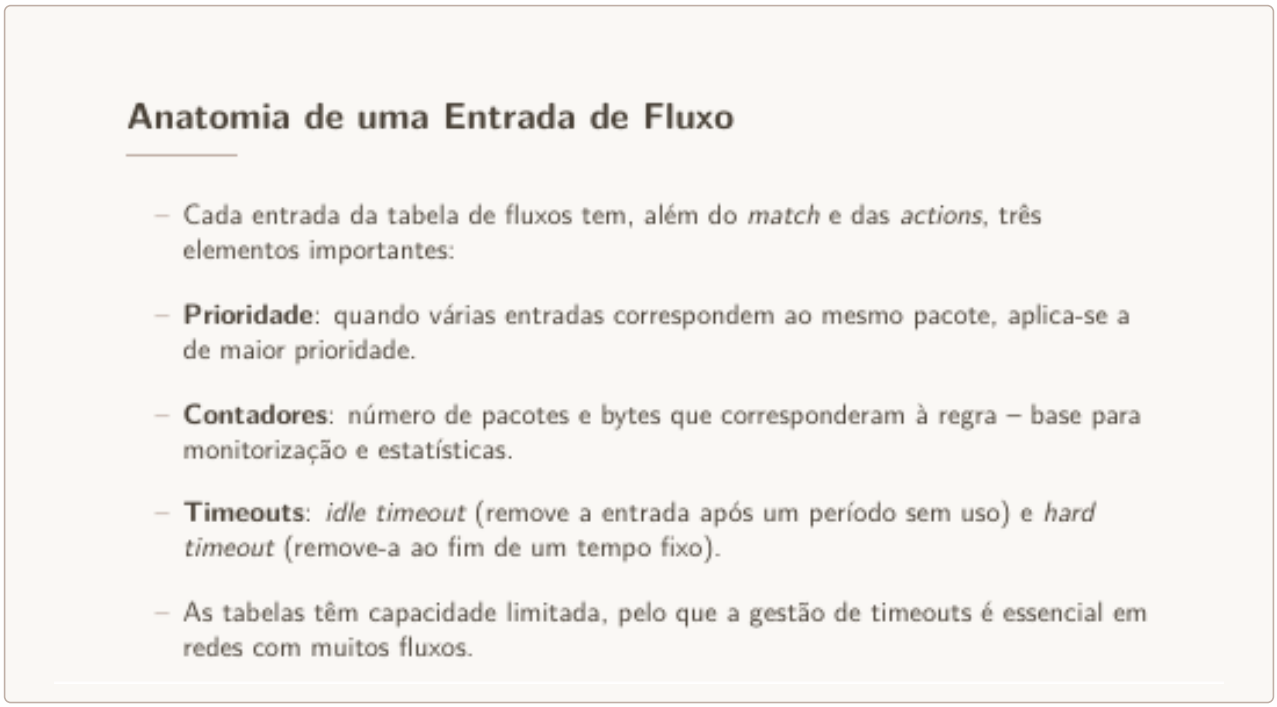
OpenFlow e Tabelas de Fluxos

- OpenFlow é o protocolo southbound mais usado para comunicação entre controller e switches.
- Cada switch mantém uma **tabela de fluxos**, com entradas do tipo *match – action*.
- **Match**: campos do pacote (IP origem/destino, porta, protocolo, ...).
- **Action**: encaminhar, descartar, reencaminhar para o controller, modificar cabeçalho.
- Quando não há regra aplicável, o pacote é enviado ao controller, que decide e instala uma nova regra.

O OpenFlow é o protocolo *southbound* mais difundido. Define as mensagens que o controller e o switch trocam e o modelo de dados que o switch expõe: uma ou mais tabelas de fluxos. Cada tabela contém entradas do tipo *match* e ação.

O *match* descreve os pacotes a que a entrada se aplica, através de campos do cabeçalho: porta de entrada, endereços MAC, endereços IP, protocolo, portas de transporte. A *action* diz o que fazer com o pacote: encaminhá-lo por uma porta, descartá-lo, enviá-lo ao controller ou modificar um cabeçalho.

Quando nenhuma entrada corresponde ao pacote, o comportamento depende da configuração. É habitual instalar uma entrada de prioridade mínima que envia o pacote ao controller, e é este mecanismo que sustenta o modelo reativo, adiante.



Para além do *match* e das ações, cada entrada de fluxo tem três elementos que a tornam gerível. A prioridade resolve o caso em que vários *matches* correspondem ao mesmo pacote: aplica-se a entrada de maior prioridade. É assim que uma regra específica, como o bloqueio de um par de hosts, se sobrepõe a uma regra genérica de encaminhamento.

Os contadores registam quantos pacotes e quantos bytes corresponderam à entrada. Constituem a base da monitorização, porque permitem saber quanto tráfego cada fluxo gera sem instalar nenhuma sonda adicional.

Os *timeouts* controlam o tempo de vida. O *idle timeout* remove a entrada quando deixa de haver tráfego durante um período, e o *hard timeout* remove-a ao fim de um tempo fixo, independentemente do uso. Como a memória das tabelas é limitada, estes mecanismos evitam que ela se encha de fluxos que já não existem.

Uma Tabela de Fluxos Concreta

Prioridade	Match	Ação	Contadores	Timeout
100	ip, src 10.0.0.1, dst 10.0.0.3	drop	12 pkt	hard 60 s
10	eth_dst 00:..:02, in_port 1	output:2	840 pkt	idle 30 s
1	eth_dst 00:..:01, in_port 2	output:1	835 pkt	idle 30 s
0	qualquer pacote (table-miss)	enviar ao controller	7 pkt	sem

Vence a regra de maior prioridade que corresponda ao pacote

A tabela mostra quatro entradas de um switch, ordenadas por prioridade. A primeira, de prioridade 100, descarta o tráfego IP entre dois endereços e tem um *hard timeout* de 60 segundos. As duas seguintes, de prioridades 10 e 1, encaminham tramas para a porta de um determinado destino, e vão expirar ao fim de 30 segundos sem uso. A última, de prioridade 0, corresponde a qualquer pacote e envia-o ao controller.

Dois pontos de leitura. Primeiro, a ordem de avaliação é dada pela prioridade e não pela posição na tabela. Segundo, a entrada de prioridade 0 funciona como rede de segurança: só é usada quando nenhuma outra corresponde, e o seu contador, que aqui indica sete pacotes, mede quantas vezes o controller foi consultado.

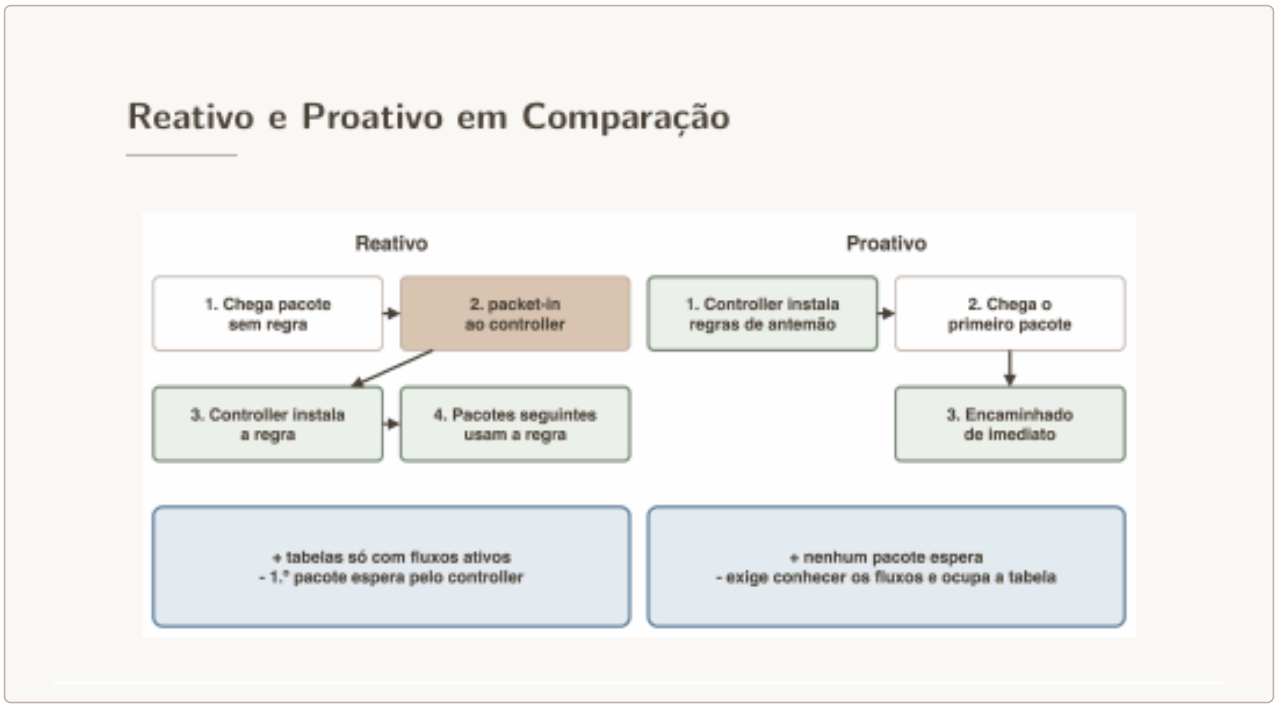
Modelo Reativo vs Proativo

- **Reativo**: as regras só são instaladas quando chega o primeiro pacote de um fluxo desconhecido (*packet-in* para o controller).
- Vantagem: as tabelas contêm apenas fluxos ativos. Desvantagem: o primeiro pacote sofre atraso e o controller pode tornar-se um estrangulamento.
- **Proativo**: as regras são instaladas antecipadamente, antes de haver tráfego.
- Vantagem: nenhum pacote espera pelo controller. Desvantagem: exige conhecer previamente os fluxos e ocupa espaço nas tabelas.
- Implementações reais combinam frequentemente as duas abordagens.

No modelo reativo, o switch arranca com uma tabela quase vazia. Quando chega o primeiro pacote de um fluxo desconhecido, é enviado ao controller numa mensagem *packet-in*. O controller decide, instala a regra e devolve o pacote. Os pacotes seguintes do mesmo fluxo já encontram a regra e não voltam a incomodar o controller.

A vantagem é que as tabelas só contêm fluxos ativos. O custo é que o primeiro pacote de cada fluxo sofre um atraso adicional e que o controller pode tornar-se um estrangulamento quando surgem muitos fluxos novos em simultâneo.

No modelo proativo, as regras são instaladas antes de haver tráfego. Nenhum pacote espera pelo controller, mas é preciso conhecer os fluxos previamente, e a tabela pode ficar ocupada com regras que nunca serão usadas. As implementações reais combinam as duas abordagens: instalam proativamente o essencial e deixam o resto para o modo reativo.



A figura coloca os dois modelos em paralelo. À esquerda, a sequência reativa tem quatro passos, e é o terceiro, a instalação da regra, que acontece depois da chegada do pacote. À direita, a sequência proativa começa pela instalação, e o primeiro pacote é encaminhado de imediato.

As caixas inferiores resumem o compromisso. O reativo poupa espaço nas tabelas e obriga o primeiro pacote a esperar. O proativo elimina a espera e exige que se saiba de antemão que fluxos vão existir. A escolha depende do tipo de rede: uma rede com padrões estáveis favorece o proativo, e uma rede muito dinâmica favorece o reativo.



O diagrama segue um pacote de h1 para h2 num switch que ainda não tem regra para ele. No passo 1, o pacote chega ao switch. No passo 2, nenhuma entrada corresponde, e o switch envia o pacote ao controller numa mensagem *packet-in*. No passo 3, o controller decide a porta de saída e responde com duas mensagens: uma *flow-mod*, que instala a regra, e uma *packet-out*, que manda o pacote em espera para a porta escolhida. No passo 4, os pacotes seguintes seguem a regra sem passar pelo controller.

Este percurso explica uma observação que se fará no laboratório. Sem controller, o switch não tem a quem perguntar e o tráfego fica retido. Com controller, o primeiro `ping` demora mais do que os seguintes, porque inclui a ida ao controller.



Há vários controllers à escolha, com públicos diferentes. O Ryu, escrito em Python, é simples e legível, e por isso foi o escolhido para o laboratório: o código de um *learning switch* cabe numa página. O OpenDaylight e o ONOS são plataformas de grande dimensão, em Java, desenhadas para operadores de telecomunicações e redes extensas, com funcionalidades de alta disponibilidade. O Faucet é um controller leve, com configuração declarativa, orientado a ambientes de produção mais pequenos.

A API *northbound*, tipicamente REST, permite que sistemas externos definam políticas sem falar OpenFlow. Na prática, é por aqui que uma plataforma de orquestração pede à rede que acompanhe a criação de uma máquina virtual.

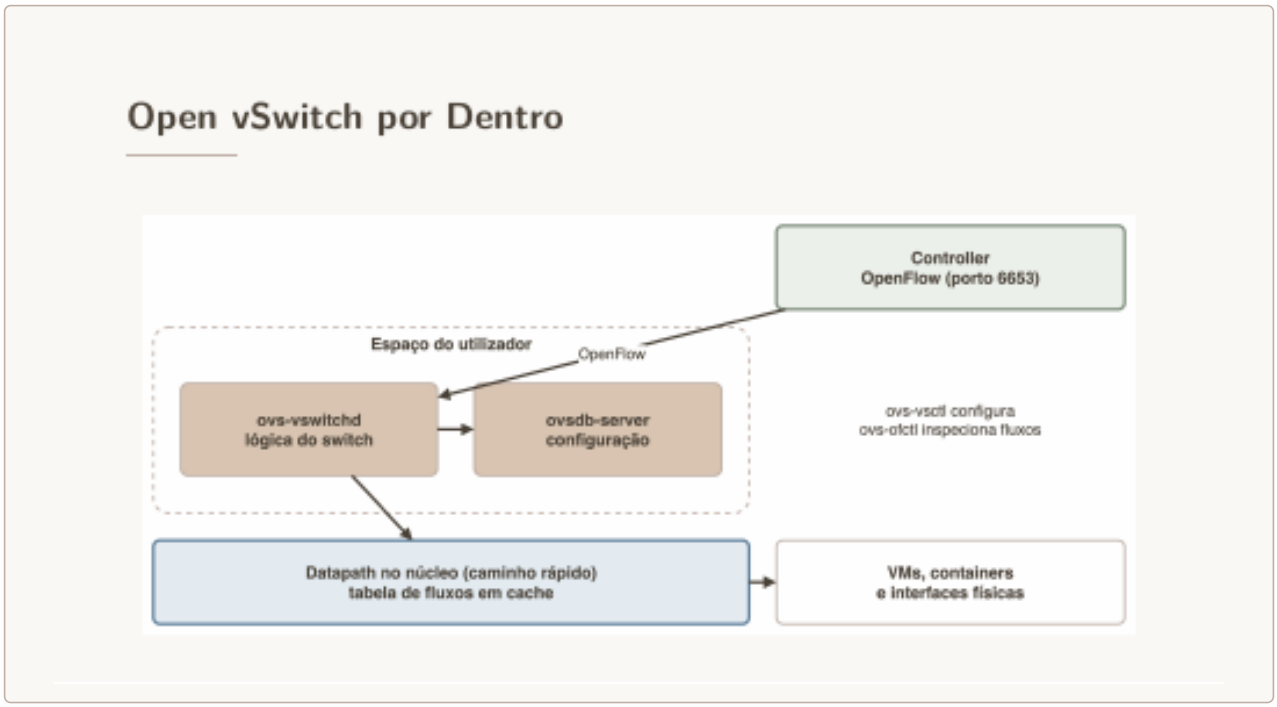
Open vSwitch (OVS)

- Open vSwitch é um switch virtual, programável via OpenFlow, amplamente usado em hipervisores e datacenters.
- Substitui a bridge de rede tradicional em hosts que correm máquinas virtuais ou containers.
- Relembrando a Aula 05: as redes que o Kubernetes usa para ligar Pods entre nós assentam frequentemente em switches programáveis deste tipo.
- Permite aplicar políticas de rede (isolamento, QoS, monitorização) de forma centralizada e programática.

O Open vSwitch, abreviadamente OVS, é um switch virtual de código aberto. Corre no anfitrião e liga as interfaces virtuais das máquinas virtuais ou dos containers entre si e à rede física. Pode ser programado via OpenFlow, o que o torna a ponte entre o modelo SDN e a infraestrutura virtualizada.

Em muitos anfitriões substitui a bridge Linux tradicional, porque oferece mais do que a simples comutação de camada 2: túneis, estatísticas por fluxo, QoS e políticas de segurança. É o motor de rede de várias plataformas de virtualização e de nuvem.

A ligação ao que já foi estudado é direta. As redes que ligam Pods entre nós, na Aula 05, assentam em mecanismos desta família, e o mesmo princípio de ligar interfaces virtuais a um switch em software é o que o Mininet reproduz no laboratório.



O OVS tem dois processos principais no espaço do utilizador. O `ovs-vswitchd` é o daemon que implementa a lógica do switch e comunica com o controller por OpenFlow. O `ovsdb-server` guarda a configuração, como as bridges e as portas, numa pequena base de dados.

O caminho rápido dos pacotes não passa pelo espaço do utilizador. Fica no núcleo do sistema, no *datapath*, que mantém em cache as decisões já tomadas. Só o primeiro pacote de cada fluxo sobe ao `ovs-vswitchd`, e os seguintes são tratados no núcleo, à velocidade da linha.

Duas ferramentas de linha de comandos acompanham o sistema. O `ovs-vsctl` configura bridges e portas, e o `ovs-ofctl` inspeciona e altera as tabelas de fluxos. São as que se usam no laboratório.

Redes Overlay: VXLAN

- Datacenters de cloud precisam de muitas redes virtuais isoladas sobre a mesma infraestrutura física.
- VLANs tradicionais limitam-se a 4094 identificadores – insuficiente à escala de cloud.
- **VXLAN** encapsula tráfego de camada 2 dentro de pacotes UDP (camada 3), criando redes overlay independentes da topologia física.
- Permite milhões de segmentos de rede lógicos e mobilidade de VMs/containers entre servidores físicos sem reconfigurar a rede física.

Um datacenter de nuvem serve muitos clientes, cada um com direito a redes isoladas. Fazê-lo com redes físicas separadas é inviável. A alternativa é construir redes virtuais sobre a mesma infraestrutura.

As VLAN tradicionais seguem esse princípio, mas o identificador tem 12 bits, o que dá 4094 redes utilizáveis. À escala de uma nuvem pública, é insuficiente. Além disso, as VLAN exigem configuração nos switches físicos ao longo do caminho.

A VXLAN encapsula tramas de camada 2 em pacotes UDP, que a rede física encaminha como qualquer outro tráfego IP. Cria assim redes lógicas independentes da topologia física, em número muito superior, e permite mover máquinas virtuais entre servidores sem reconfigurar a rede subjacente.

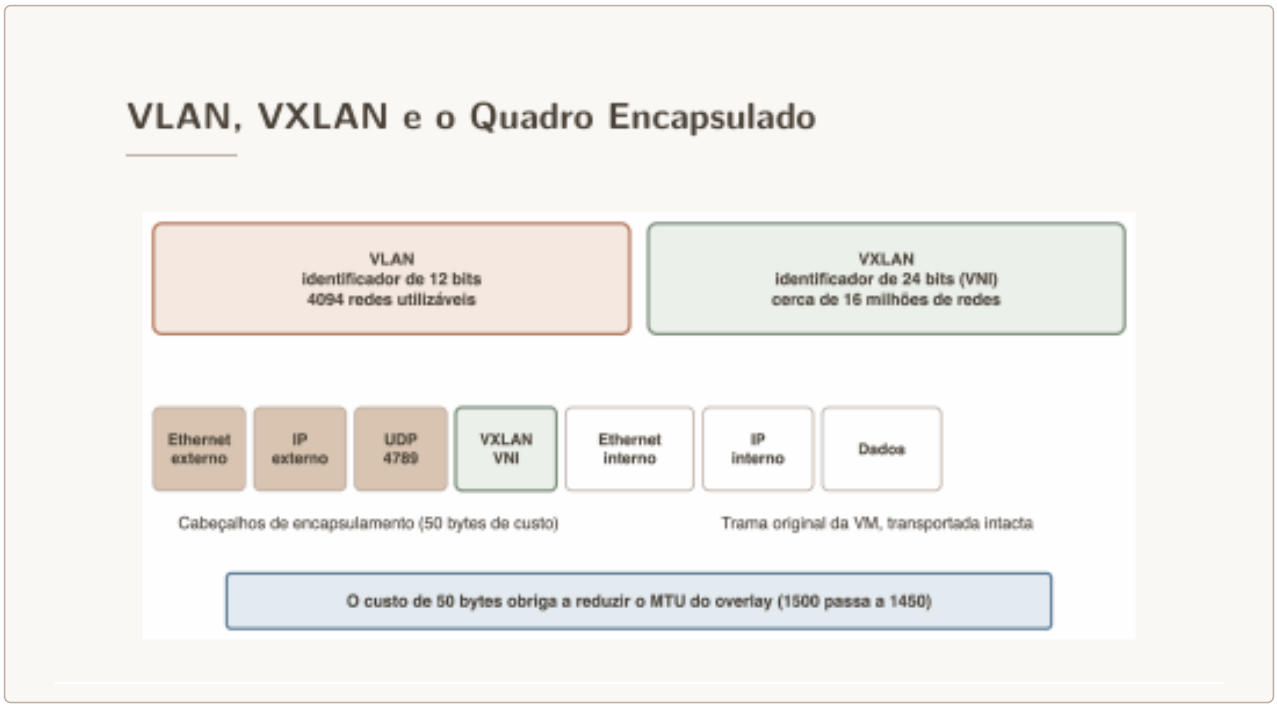
Encapsulamentos de Overlay

- **VXLAN**: encapsula tramas de camada 2 em UDP; identificador de 24 bits (VNI), permitindo cerca de 16 milhões de segmentos.
- **GRE** e **Geneve**: alternativas de encapsulamento; o Geneve foi desenhado para ser extensível através de opções variáveis.
- Terminologia VXLAN: o **VTEP** (VXLAN Tunnel Endpoint) é o ponto onde o encapsulamento é aplicado e removido.
- Custo: o cabeçalho adicional reduz o espaço útil de cada pacote, obrigando a ajustar o MTU da rede subjacente.
- Distinção útil: a rede física subjacente designa-se **underlay**; a rede virtual construída sobre ela, **overlay**.

A VXLAN identifica cada rede lógica por um VNI, um identificador de 24 bits, o que permite cerca de 16 milhões de segmentos. O ponto em que a trama é encapsulada e desencapsulada chama-se VTEP, *VXLAN Tunnel Endpoint*. Pode ser um switch virtual no anfitrião ou um equipamento físico.

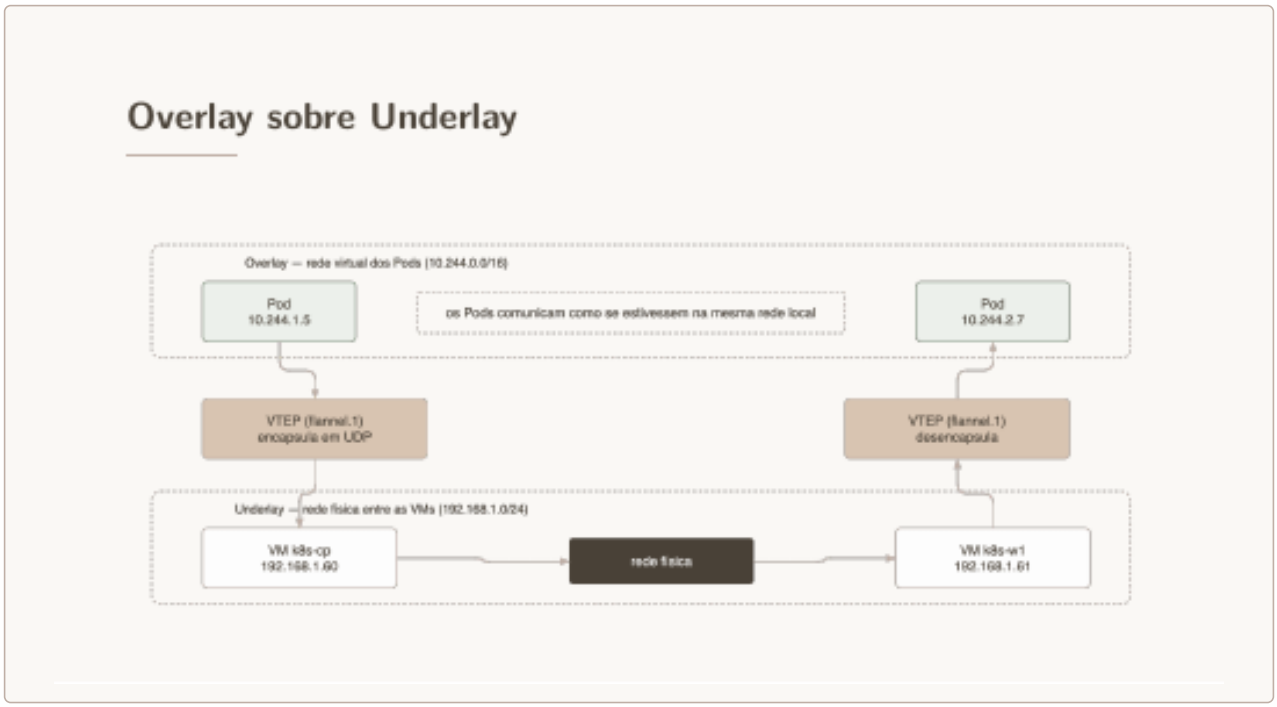
Existem alternativas. O GRE é um encapsulamento mais simples e antigo. O Geneve foi desenhado para ser extensível, com campos opcionais de comprimento variável, e é usado por vários sistemas de rede virtual mais recentes.

Todo o encapsulamento tem um custo. Os cabeçalhos adicionais ocupam espaço em cada pacote, pelo que o MTU útil do overlay é inferior ao da rede física, ou então esta tem de aceitar pacotes maiores. Convém ainda fixar a terminologia: a rede física é o *underlay*, e a rede virtual construída sobre ela é o *overlay*.



Na parte superior comparam-se os espaços de endereçamento: 12 bits na VLAN e 24 bits na VXLAN. A diferença de 4094 para cerca de 16 milhões é a razão pela qual a VXLAN domina em ambientes de nuvem.

Na parte inferior mostra-se a estrutura de um pacote VXLAN. A trama original da máquina virtual, com os seus cabeçalhos Ethernet e IP e os dados, viaja intacta. À volta dela são acrescentados quatro cabeçalhos: um Ethernet e um IP externos, que dizem como chegar ao VTEP de destino, um UDP, normalmente com o porto 4789, e o cabeçalho VXLAN, que transporta o VNI. Somados, ocupam 50 bytes. Por isso, numa rede física com MTU de 1500, o overlay anuncia habitualmente 1450.



A figura separa dois níveis que convém nunca confundir. Na parte inferior está o *underlay*: as máquinas com os seus endereços reais, ligadas pela rede física. Na parte superior está o *overlay*: a rede virtual dos Pods, com endereços próprios, que não existe fisicamente.

O percurso lê-se de cima para baixo no lado esquerdo e de baixo para cima no lado direito. O Pod de origem entrega o pacote ao VTEP local, que o encapsula em UDP. O pacote encapsulado atravessa a rede física como tráfego comum entre as duas máquinas. No destino, o VTEP desencapsula-o e entrega a trama original ao Pod. Do ponto de vista dos Pods, é como se estivessem na mesma rede local.

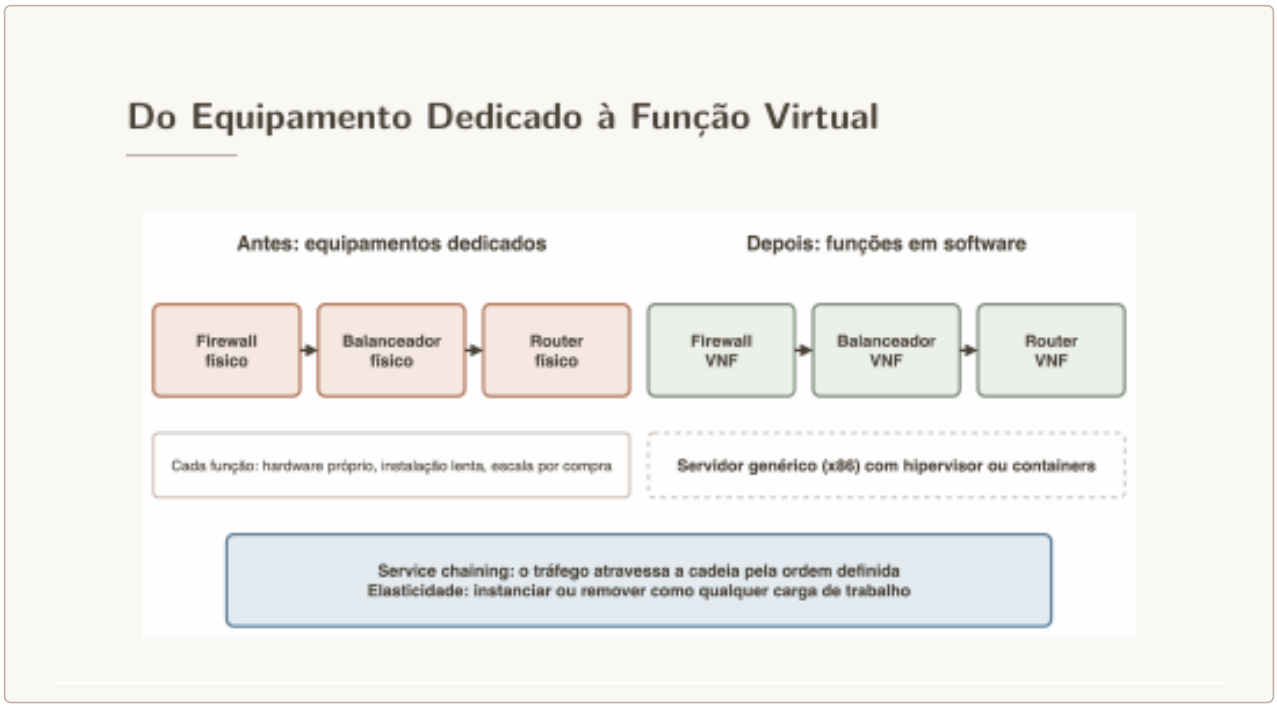
Network Functions Virtualization (NFV)

- Conceito complementar da SDN: substituir equipamento de rede dedicado por software em servidores genéricos.
- Exemplos de funções virtualizadas: firewalls, balanceadores de carga, routers, gateways de VPN.
- **SDN vs NFV**: a SDN trata de *como* o tráfego é encaminhado; a NFV trata de *onde* correm as funções de rede.
- **Service chaining**: encadear várias funções virtualizadas pelas quais o tráfego deve passar sequencialmente.
- Vantagem central: elasticidade – instanciar ou remover funções de rede tão facilmente como qualquer outra carga de trabalho.

A virtualização de funções de rede, ou NFV, é complementar da SDN. Em vez de comprar um equipamento dedicado para cada função, executa-se a função como software em servidores genéricos. Firewalls, balanceadores de carga, routers e gateways de VPN são os exemplos habituais.

A distinção entre as duas é útil. A SDN diz respeito a *como* o tráfego é encaminhado, e a NFV diz respeito a *onde* correm as funções que o tratam. Podem usar-se separadamente, mas combinam bem: o controller encaminha o tráfego pelas funções virtuais certas.

Essa sequência chama-se *service chaining*: o tráfego atravessa, por exemplo, uma firewall, depois um balanceador e por fim o servidor. A vantagem principal é a elasticidade, porque instanciar ou remover uma função passa a ser tão simples como lançar qualquer outra carga de trabalho.



À esquerda, três equipamentos físicos em cadeia, cada um com hardware próprio. Acrescentar capacidade a uma função implica comprar, instalar e ligar mais equipamento, e a cadeia é fixada pela cablagem. À direita, as mesmas três funções passam a ser software, as VNF, a correr num servidor genérico, sob um hipervisor ou em containers.

A caixa inferior resume os ganhos. A cadeia de serviço passa a ser definida por configuração e pode ser alterada sem tocar na cablagem. A capacidade acompanha a procura, porque cada função pode ser replicada ou removida em segundos. O preço a pagar é o desempenho: o software em servidores genéricos precisa, em tráfego muito intenso, de técnicas de aceleração para igualar o hardware dedicado.

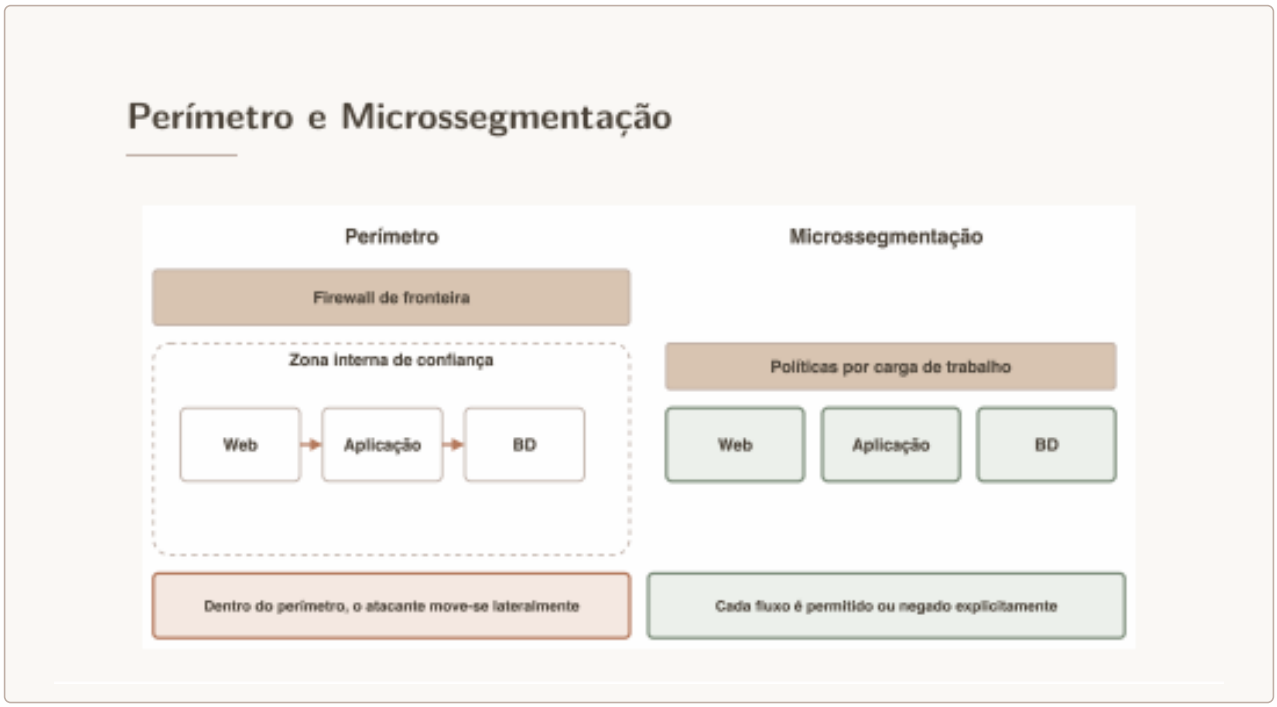
Microssegmentação e Políticas de Rede

- Modelo tradicional: um perímetro fortificado (firewall na fronteira) e confiança implícita no interior da rede.
- Problema: uma vez ultrapassado o perímetro, um atacante move-se lateralmente com pouca resistência.
- **Microssegmentação**: aplicar políticas de segurança ao nível de cada carga de trabalho, e não apenas na fronteira.
- A programabilidade da SDN torna isto viável em escala: as políticas são definidas centralmente e aplicadas em cada switch virtual.
- No Kubernetes, o recurso **NetworkPolicy** concretiza esta ideia, restringindo que Pods podem comunicar entre si.

O modelo clássico de segurança de rede assenta num perímetro. Uma firewall na fronteira separa o exterior, hostil, do interior, considerado de confiança. Dentro do perímetro, as máquinas comunicam livremente.

O problema é o que acontece depois de o perímetro ser ultrapassado. Basta comprometer uma máquina para que o atacante se mova lateralmente, de máquina em máquina, sem encontrar resistência. Num datacenter moderno, onde as cargas de trabalho são muitas e dinâmicas, o interior é grande e a confiança implícita é um risco.

A microssegmentação aplica políticas ao nível de cada carga de trabalho. Cada fluxo é permitido ou negado explicitamente, e o que não estiver previsto é recusado. A programabilidade da SDN torna isto exequível: as políticas definem-se centralmente e são aplicadas em cada switch virtual. No Kubernetes, o recurso NetworkPolicy concretiza a ideia, ao restringir que Pods podem comunicar com quais.



À esquerda, o perímetro. A firewall protege a fronteira, e dentro dela as três camadas, web, aplicação e base de dados, comunicam sem restrição. As setas a vermelho mostram o caminho que um atacante pode seguir depois de entrar pela primeira.

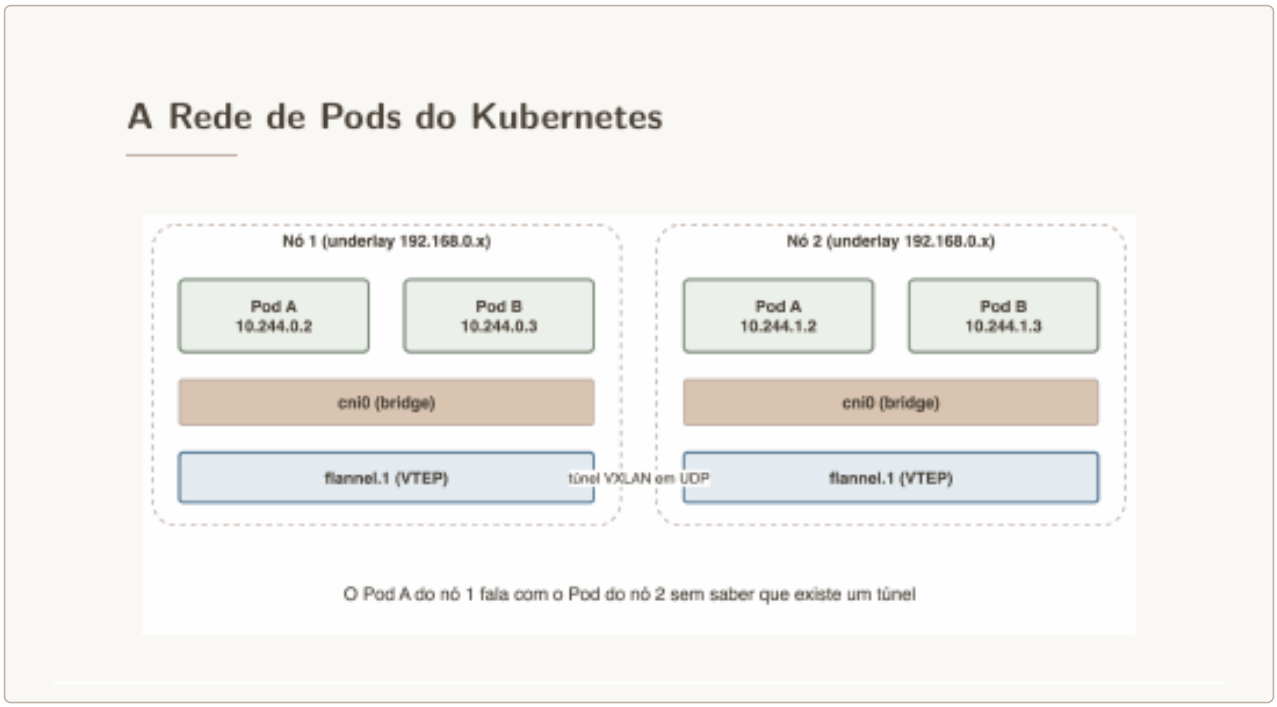
À direita, a microssegmentação. Cada carga de trabalho tem a sua política, e a comunicação entre elas é limitada ao que é necessário. A camada web pode falar com a aplicação, e a aplicação com a base de dados, mas a web não chega à base de dados. Um compromisso da camada web já não dá acesso direto aos dados.



O Proxmox VE liga as máquinas virtuais através de bridges Linux, como a `vmbr0`, e foi o que se usou nos guiões anteriores. Uma bridge é um switch de camada 2 em software, simples e eficaz, mas sem programabilidade.

O Proxmox dispõe também de um módulo SDN. Permite definir *zonas*, entre as quais a VXLAN, e *VNets* sobre a infraestrutura física, de modo que várias máquinas em servidores diferentes partilhem a mesma rede lógica. É a aplicação direta do conceito de overlay a um ambiente de virtualização.

No Kubernetes, o plugin CNI desempenha o papel da camada de rede programável. O Flannel, por omissão, cria um overlay entre os nós com encapsulamento VXLAN. Ou seja, um cluster como o da Aula 05 já usa os conceitos desta aula, ainda que não se tenha falado deles.



O Kubernetes exige que cada Pod tenha o seu endereço IP e que qualquer Pod possa comunicar com qualquer outro, mesmo noutro nó. Não diz como. Quem o resolve é o plugin CNI.

A figura mostra o caso do Flannel com VXLAN. Em cada nó, os Pods ligam-se a uma bridge local, a `cni0`, e a interface `flannel.1` faz de VTEP. Quando o Pod A do nó 1 envia um pacote ao Pod do nó 2, o VTEP do nó 1 encapsula-o em UDP e envia-o pela rede física para o nó 2, onde o outro VTEP o desencapsula. Cada nó recebe uma sub-rede própria da rede dos Pods, o que permite saber para que nó encaminhar cada destino.

Para o Pod, nada disto é visível. Comunica com o endereço do outro Pod como se estivesse na mesma rede local.



Por omissão, o Kubernetes permite que qualquer Pod comunique com qualquer outro. A NetworkPolicy altera isso. Quando uma política seleciona um Pod, apenas é permitido o tráfego que a política enumera, e o restante é recusado.

No exemplo, o frontend pode contactar o backend no porto TCP 8080, e o backend pode contactar a base de dados no porto 5432. Não existe nenhuma regra que permita ao frontend chegar à base de dados, pelo que essa ligação é negada. Cada política seleciona os Pods por etiquetas, tal como o Service da Aula 05.

Há uma condição que convém conhecer. A NetworkPolicy é apenas uma declaração: quem a aplica é o plugin CNI. Plugins como o Calico ou o Cilium aplicam-na. O Flannel, sozinho, não a aplica, e as políticas criadas num cluster assim não têm efeito.

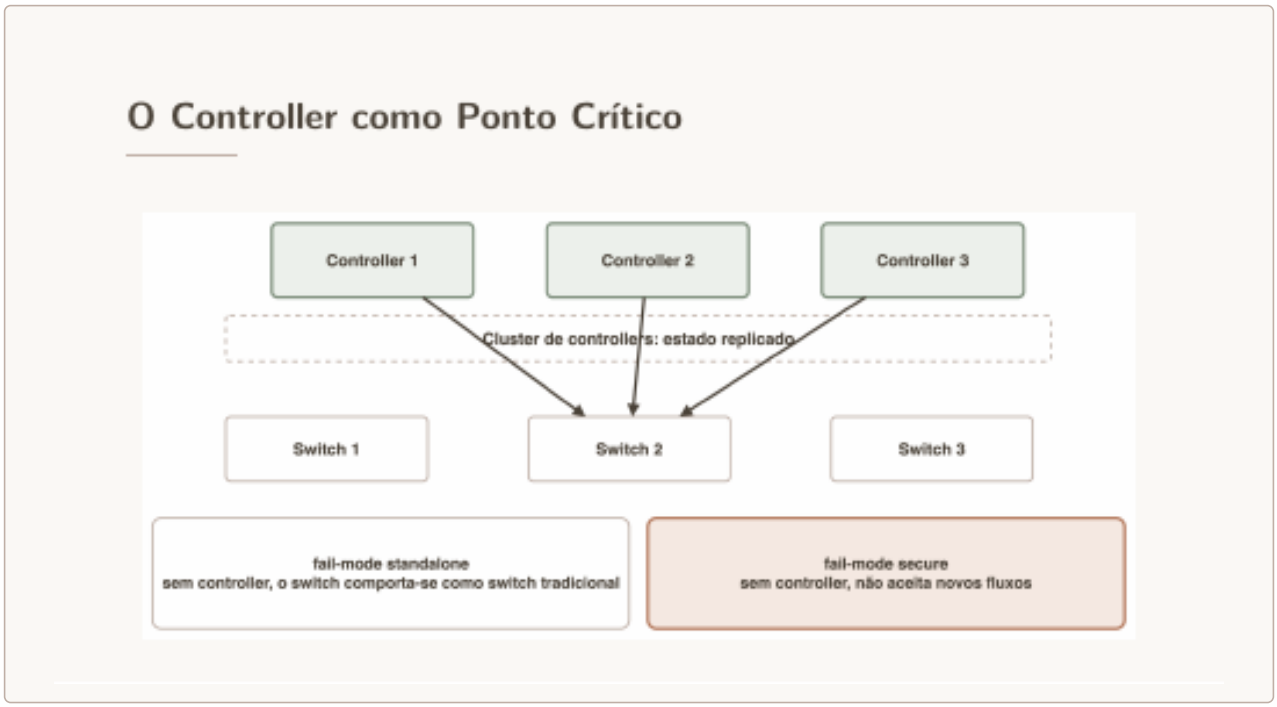
Vantagens e Desafios da Centralização

- **Vantagens**: visão global da rede, políticas consistentes, automação, programabilidade (rede tratada como software).
- **Desafio**: o controller é um ponto crítico – se falhar, pode comprometer toda a rede.
- Mitigação: controllers em cluster (alta disponibilidade), replicando o padrão já visto para outros serviços na Aula 08/09.
- **Segurança**: o canal controller-switch tem de ser protegido, pois concentra o poder de decisão de toda a rede.

A centralização traz vantagens claras. Há uma visão global da rede, as políticas são consistentes porque são definidas num só lugar, a automação torna-se natural, e a rede passa a ser tratada como software, com tudo o que isso permite: versões, revisão, testes.

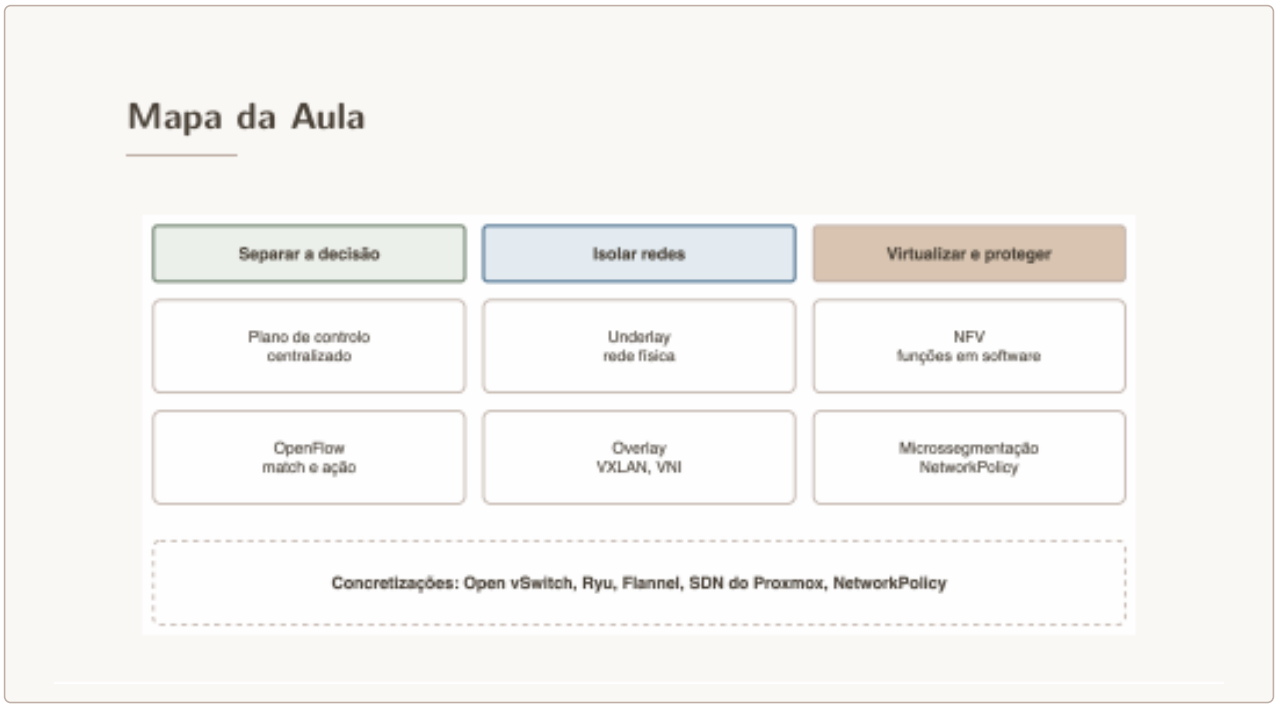
O reverso é o controller ser um ponto crítico. Se falhar, a rede pode ficar sem capacidade de decidir, e se for comprometido, o atacante controla o encaminhamento de toda a rede. A mitigação da disponibilidade é replicar o controller em cluster, o mesmo padrão que será estudado nas Aulas 08 e 09 para outros serviços.

A segurança do canal entre controller e switches merece atenção especial, porque concentra o poder de decisão. Deve ser autenticado e cifrado, e o acesso ao controller deve ser tão restrito como o acesso ao próprio datacenter.



A figura mostra um cluster de três controllers, com o estado replicado entre si, ligados ao mesmo switch. Se um falhar, os outros continuam a responder, e o switch mantém sempre um controller com quem falar.

As caixas inferiores tratam o caso extremo, em que o switch perde todos os controllers. No Open vSwitch isto define-se pelo *fail-mode*. No modo *standalone*, o switch passa a comportar-se como um switch tradicional e mantém o tráfego a fluir, com a perda da programabilidade. No modo *secure*, não aceita novos fluxos, e apenas as regras já instaladas continuam a ser aplicadas. A escolha é um compromisso entre disponibilidade e controlo.



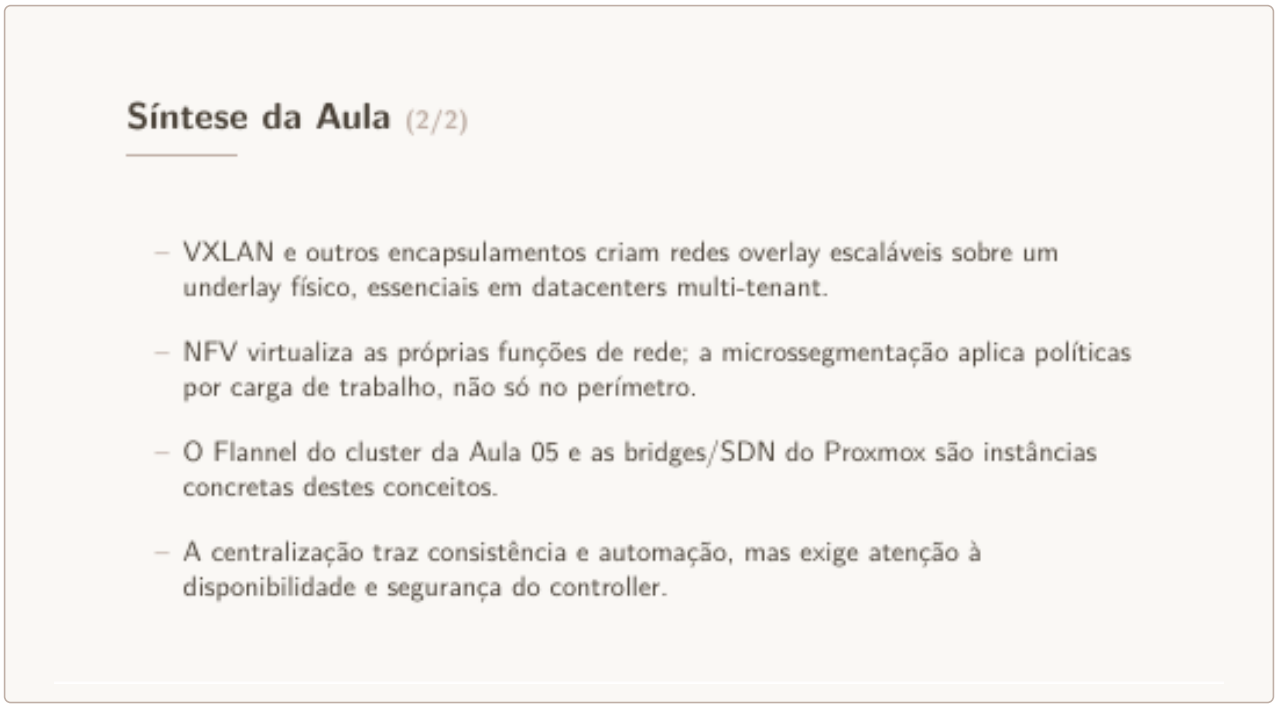
O mapa reúne os conceitos da aula em três ideias. A primeira é separar a decisão: o plano de controlo centralizado e o OpenFlow, com o seu modelo de *match* e ação. A segunda é isolar redes: a rede física como *underlay* e as redes lógicas como *overlay*, identificadas por um VNI. A terceira é virtualizar e proteger: funções de rede em software e políticas por carga de trabalho.

A linha inferior lista as concretizações encontradas: o Open vSwitch, o Ryu, o Flannel, o módulo SDN do Proxmox e a NetworkPolicy. Cada uma é uma instância de uma das três ideias, e saber a que ideia pertence ajuda a perceber o que faz.



A síntese resume a primeira metade da matéria. A SDN separa o plano de controlo, centralizado no controller, do plano de dados, distribuído pelos switches. O OpenFlow é o protocolo *southbound* típico e assenta em tabelas de fluxos com entradas *match* e ação, com prioridade, contadores e *timeouts*.

O Open vSwitch concretiza este modelo em hipervisores e em hosts de containers. O controller pode instalar as regras de forma reativa, quando surge o primeiro pacote, ou de forma proativa, antes de haver tráfego, e as implementações reais misturam as duas.



A segunda parte da síntese recolhe o resto. A VXLAN e os restantes encapsulamentos criam redes overlay escaláveis sobre um underlay físico, o que é essencial em datacenters partilhados por muitos clientes. A NFV virtualiza as próprias funções de rede, e a microssegmentação aplica políticas a cada carga de trabalho, e não apenas no perímetro.

O cluster da Aula 05 e as bridges e o módulo SDN do Proxmox são instâncias concretas destes conceitos. A centralização oferece consistência e automação, mas exige que se cuide da disponibilidade e da segurança do controller. A Aula 07 prolonga esta lógica: a configuração, que aqui se centralizou para a rede, passa a ser descrita como código para toda a infraestrutura.

Obrigada.

Questões e comentários

Este é o espaço para questões e comentários. Sugere-se que o estudante, antes do laboratório, consiga explicar com palavras suas três coisas: porque é que um switch sem controller não encaminha tráfego, o que acontece, passo a passo, ao primeiro pacote de um fluxo no modelo reativo, e qual a diferença entre o underlay e o overlay.

O laboratório da Aula 06 torna observáveis as duas primeiras, com o Mininet, o Open vSwitch e o Ryu, e a terceira, com um túnel VXLAN criado entre dois namespaces de rede.